Curs 2: Lucrul cu colecții, funcții, module și pachete Python, pachetul NumPy

Collection comprehension

Pornind de la o colecție (cel mai frecvent caz: de la o listă) se poate crea o altă colecție, folosind *collection comprehension* - în esență o ciclare:

In [1]:
```
lista_numere = [1, 3, 6, 21, 22, 32, 33]
lista_patrate = [i * i for i in lista_numere]
print(lista_patrate)
```

```
[1, 9, 36, 441, 484, 1024, 1089]
```

Opțional, la fiecare pas al iterării se poate lua în considerare o condiție `if` inline:

In [2]:
```
lista_patrate_doar_numere_pare = [i * i for i in lista_numere if i % 2 == 0]
print(lista_patrate_doar_numere_pare)
```

```
[36, 484, 1024]
```

... sau se folosește și `else` pe lângă `if`:

In [3]:
```
lista_patrate_sau_cuburi = [i**2 if i % 2 == 0 else i**3 for i in lista_numere]
print(lista_patrate_sau_cuburi)
```

```
[1, 27, 36, 9261, 484, 1024, 35937]
```

Exercițiu: dacă o listă are ca elemente alte liste de numere, cum se poate determina lista 'flattened'? De exemplu, pentru lista `a1 = [[1, 2], [3, 4, 5], [10]]` se dorește rezultatul `a2 = [1, 2, 3, 4, 5, 10]`.

In [4]:
```
a1 = [[1, 2], [3, 4, 5], [10]]
a2 = [item for sublist in a1 for item in sublist]
print(a2)
```

```
[1, 2, 3, 4, 5, 10]
```

Comprehension se poate folosi și peste alte tipuri de colecții: de exemplu, putem pleca de la o listă și producem un dicționar:

In [5]:
```
lista_numere = [1, 3, 6, 21, 22, 32, 33]
dictionar_numere_si_patrate = {i: i**2 for i in lista_numere}
print(dictionar_numere_si_patrate)
```

```
{1: 1, 3: 9, 6: 36, 21: 441, 22: 484, 32: 1024, 33: 1089}
```

... sau liste de tupluri:

In [6]:
```
# produs cartezian
colours = ["red", "green", "yellow", "blue"]
things = ["house", "car", "tree"]
produs_cartezian = [(colour, thing) for colour in colours for thing in things]
print(produs_cartezian)
assert len(produs_cartezian) == len(colours) * len(things)
```

```
[('red', 'house'), ('red', 'car'), ('red', 'tree'), ('green', 'house'), ('green', 'c
ar'), ('green', 'tree'), ('yellow', 'house'), ('yellow', 'car'), ('yellow', 'tree'),
('blue', 'house'), ('blue', 'car'), ('blue', 'tree')]
```

Mai jos sunt câteva exemple de utilizare de comprehension peste colecții.

In [7]:
```
# Conversie de temperaturi din Celsius in Fahrenheit: valoarea in Fahrenheit se obt
grade_celsius = [-20, -10, 0, 5, 23, 35]
grade_fahrenheit = [1.8 * gc + 32 for gc in grade_celsius]
print(grade_fahrenheit)
```

```
[-4.0, 14.0, 32.0, 41.0, 73.4, 95.0]
```

In [8]:
```
# Suma patratelor numerelor de la 1 la 20
print(sum([x**2 for x in range(1, 21)]))
```

```
2870
```

In [9]:
```
# Dintr-o lista de cuvinte se mentin doar cele care nu fac parte dintr-o multime (s
# stop_words e definita ca o multime (set) pentru a permite cautarea rapida a unui
stop_words = {"a", "and", "for", "from", "in", "is", "of", "there", "this"}
# etc; lista completa poate fi gasita la https://www.ranks.nl/stopwords

paragraph_list = [
    "Stopword",
    "filtering",
    "is",
    "a",
    "common",
    "step",
    "in",
    "preprocessing",
    "text",
    "for",
    "various",
    "purposes",
    "This",
    "is",
    "a",
    "list",
    "of",
    "several",
    "different",
    "stopword",
    "lists",
    "extracted",
    "from",
    "various",
    "search",
```

```
    "engines",
    "libraries",
    "and",
    "articles",
    "There",
    "is",
    "a",
    "surprising",
    "number",
    "of",
    "different",
    "lists",
]
print("Initial:", paragraph_list)
filtered = [cuvant for cuvant in paragraph_list if cuvant.lower() not in stop_words
print("\nDupa filtrare:", filtered)
```

```
Initial: ['Stopword', 'filtering', 'is', 'a', 'common', 'step', 'in', 'preprocessin
g', 'text', 'for', 'various', 'purposes', 'This', 'is', 'a', 'list', 'of', 'severa
l', 'different', 'stopword', 'lists', 'extracted', 'from', 'various', 'search', 'eng
ines', 'libraries', 'and', 'articles', 'There', 'is', 'a', 'surprising', 'number',
'of', 'different', 'lists']

Dupa filtrare: ['Stopword', 'filtering', 'common', 'step', 'preprocessing', 'text',
'various', 'purposes', 'list', 'several', 'different', 'stopword', 'lists', 'extract
ed', 'various', 'search', 'engines', 'libraries', 'articles', 'surprising', 'numbe
r', 'different', 'lists']
```

Funcții

Funcțiile sunt de trei feluri:

- Funcții deja definite în limbajul Python, cum ar fi `len()`, `print()`
- Funcții definite de utilizator
- Lambda funcții

O funcție se definește folosind cuvântul cheie `def`. Blocul de instrucțiuni ce definește corpul funcției este indentat. O funcție returnează întotdeauna un singur obiect: valoarea dată de `return` sau, dacă `return` lipsește, `None`. Pentru a „returna mai multe valori" se returnează un tuplu, de exemplu `return a, b`, echivalent cu `return (a, b)`, care poate fi apoi despachetat: `x, y = f()`.

Funcții definite de utilizator

Urmează câteva exemple de funcții definite de utilizator, cu comentarii:

In [10]:
```
def hello():
    print("Salutare")


hello()
```

```
Salutare
```

In [11]:
```
def hello_with_name(nume):
    """
    Functia preia un argument si afiseaza mesajul: Salutare urmat de valoarea argum
    Functia returneaza argumentul cu litere mari.
    :param nume: numele care se cere afisat
    :return: sirul din :param nume: cu litere mari
    """
    print("Salutare " + nume)
    return nume.upper()


nume = "Natalia"
nume_litere_mari = hello_with_name(nume)
print(nume_litere_mari)
help(hello_with_name)
print(hello_with_name.__doc__)
```

```
Salutare Natalia
NATALIA
Help on function hello_with_name in module __main__:

hello_with_name(nume)
    Functia preia un argument si afiseaza mesajul: Salutare urmat de valoarea argume
ntului.
    Functia returneaza argumentul cu litere mari.
    :param nume: numele care se cere afisat
    :return: sirul din :param nume: cu litere mari


Functia preia un argument si afiseaza mesajul: Salutare urmat de valoarea argumentul
ui.
Functia returneaza argumentul cu litere mari.
:param nume: numele care se cere afisat
:return: sirul din :param nume: cu litere mari
```

In [12]:
```
# exemplu de functie care returneaza mai multe valori simultan
# rezultatul este un tuplu cu doua valori
def min_max(a, b):
    if a < b:
        return a, b
    else:
        return b, a


x, y = 20, 10
min_2, max_2 = min_max(x, y)
print("Minimul este:", min_2, "; maximul este:", max_2)
```

```
Minimul este: 10 ; maximul este: 20
```

In [13]:
```
# parametrii se pot da prin numele lor urmat de egal si valoarea efectiva
min_max(a=5, b=14)
```

Out[13]: (5, 14)

In [14]:
```
min_max(b=3, a=20)
```

Out[14]: (3, 20)

Pot exista parametri cu valori implicite, precizați la finalul listei de parametri formali:

In [15]:
```
def greet(name, msg="buna dimineata!"):
    """
    This function greets the person with the provided message.

    If message is not provided, it defaults to "buna dimineata!"
    :param name: Name of the guy to be greeted
    :param msg: a message shown as greeting. It defaults to "buna dimineata"
    """

    print("Salut", name + ", " + msg)


greet("Maria")
greet("George", "ce mai faci?")
# echivalent: greet(name="George",msg="ce mai faci?")
```

```
Salut Maria, buna dimineata!
Salut George, ce mai faci?
```

Capcană: valori implicite mutabile

Valoarea implicită a unui parametru se calculează o singură dată, atunci când se execută instrucțiunea `def`, și nu la fiecare apel. Dacă această valoare este un obiect mutabil (listă, dicționar, mulțime), toate apelurile care nu precizează parametrul folosesc același obiect. Dacă la un apel se modifică valoarea implicită, ea rămâne vizibilă și în apelurile următoare:

In [16]:
```
# varianta gresita: lista implicita este creata o singura data, la definirea functi
def adauga_nota(nota, note=[]):
    note.append(nota)
    return note


print(adauga_nota(10))
print(adauga_nota(7))  # ne-am astepta la [7]
print(adauga_nota(9))

# valoarea implicita este pastrata in obiectul functie si se vede cum s-a modificat
print(adauga_nota.__defaults__)
```

```
[10]
[10, 7]
[10, 7, 9]
([10, 7, 9],)
```

Lista implicită este păstrată în obiectul funcție (atributul `__defaults__`) și este aceeași la fiecare apel. Soluția uzuală este ca valoarea implicită să fie `None`, iar obiectul mutabil să fie

creat în corpul funcției:

In [17]:
```
# varianta corecta: None ca valoare implicita, lista se creeaza la fiecare apel
def adauga_nota(nota, note=None):
    if note is None:
        note = []
    note.append(nota)
    return note


print(adauga_nota(10))
print(adauga_nota(7))
print(adauga_nota(9, [8, 6]))
```

```
[10]
[7]
[8, 6, 9]
```

De reținut:

- ca valori implicite se folosesc doar obiecte imuabile: `None`, numere, șiruri de caractere, `True` / `False`, tupluri;
- se evită `[]`, `{}`, `set()` și, în general, valorile obținute printr-un apel de funcție, care s-ar calcula o singură dată (de exemplu `datetime.now()`);
- când e nevoie de o listă sau de un dicționar implicit, se scrie `parametru=None`, iar obiectul se creează în funcție.

Detalii: Default Argument Values în tutorialul Python.

Funcții cu număr arbitrar de argumente

Putem avea un parametru care să permită un număr variabil de valori trimise la apel; acest tip de parametru se scrie cu `*` urmat de numele parametrului formal (de exemplu: `*args`).

In [18]:
```
# Functie cu numar arbitrar de argumente
def greet(*names, msg="buna dimineata!"):
    for name in names:
        print("Servus", name + ", " + msg)


greet("Dan", "John", "Mary")
greet("Dan", "John", "Mary", msg="ce mai faci?")
```

```
Servus Dan, buna dimineata!
Servus John, buna dimineata!
Servus Mary, buna dimineata!
Servus Dan, ce mai faci?
Servus John, ce mai faci?
Servus Mary, ce mai faci?
```

Se pot defini funcții care să manipuleze un număr variabil de parametri dați la apel sub formă de `nume_parametru=valoare_parametru`; denumirea tradițională este `kwargs`

(keyword arguments), numele parametrului se prefixează cu `**`:

In [19]:
```
def demo_kwargs(**kwargs):
    print(kwargs)


demo_kwargs(fruits="apples", quantity="3", measurement_unit="kg")
```

```
{'fruits': 'apples', 'quantity': '3', 'measurement_unit': 'kg'}
```

Iterarea peste perechile din `kwargs` se face ținând cont că acesta este un dicționar:

In [20]:
```
def demo_kwargs_iter(**kwargs):
    for key, value in kwargs.items():
        print(key, value)


demo_kwargs_iter(fruits="apples", quantity="3", measurement_unit="kg")
```

```
fruits apples
quantity 3
measurement_unit kg
```

Același efect se obține prin despachetarea de dicționare, folosind `**`:

In [21]:
```
dictionar_argumente = {"fruits": "apples", "quantity": "3", "measurement_unit": "kg
demo_kwargs(**dictionar_argumente)
```

```
{'fruits': 'apples', 'quantity': '3', 'measurement_unit': 'kg'}
```

Ordonarea parametrilor declarați într-o funcție este:

1. parametri formali preluați prin poziție
2. `*args`
3. parametri cu valori implicite
4. `**kwargs`

```
def example2(arg_1, arg_2, *args, kw_1="shark", kw_2="blobfish",
**kwargs):
```

Lambda funcții

Se pot defini funcții anonime (sau: lambda funcții), conținând o expresie, pentru care nu se consideră necesară definirea unor funcții separate. O lambda funcție poate să preia oricâte argumente și calculează o expresie pe baza lor. Lambda funcțiile pot accesa atât parametrii trimiși, cât și variabilele globale. Se va omite cuvântul `return`, expresia calculată este cea care se returnează automat.

In [22]:
```
suma = lambda x, y: x + y
print(suma(3, 4))
```

```
7
```

In [23]:
```
# lambda functie pentru filtrare via functia filter
lista_30 = list(range(30))
lista_filtrata = list(filter(lambda x: x % 3 == 0, lista_30))
print(lista_filtrata)
```

```
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27]
```

In [24]:
```
# lambda functie pentru sortare:
sorted([-1, -2, -3, 2, 3, 4, -5, 6, 7, 8, 9], key=lambda x: x**2)
```

Out[24]:
```
[-1, -2, 2, -3, 3, 4, -5, 6, 7, 8, 9]
```

Funcții callback

Numele unei funcții reprezintă un obiect funcție:

In [25]:
```
def sum_2(x, y):
    return x + y


def dif_2(x, y):
    return x - y


# reprezentarea prin functia repr este implementata implicit prin adresa de memorie
print(dif_2)
```

```
<function dif_2 at 0x000001BD5DD29C60>
```

Putem folosi acest mecanism pentru a trimite funcții ca parametri într-o altă funcție:

In [26]:
```
def complex_operation(x, y, to_be_called):
    return to_be_called(x, y)


print(complex_operation(2, 3, sum_2))
print(complex_operation(2, 3, dif_2))
```

```
5
-1
```

Adnotări de tipuri

Se pot adnota variabilele, parametrii de funcții, tipul de retur al funcțiilor. Operația de type annotations este opțională și este destinată a mări lizibilitatea codului. Adnotările sunt definite în documentul de Python Enhancement Proposal PEP484.

În afară de cod mai ușor de citit, un alt beneficiu este analiza statică de cod, de exemplu în Visual Studio Code, PyCharm sau MyPy, sau suport îmbunătățit pentru code completion.

Adnotările se precizează după numele variabilei, urmat de două puncte și denumirea tipului:

In [27]:
```
age: int = 21
name: str = "Guido van Rossum"
```

Adnotarea nu inhibă în niciun fel utilizarea variabilelor, sau schimbarea în instrucțiunile următoare a tipului lor:

In [28]:
```
age = 21.5
```

Pentru adnotarea parametrilor și a tipului funcțiilor se poate urmări exemplul de mai jos:

In [29]:
```
def f(name: str, age: int) -> str:
    return "Salut " + name + ", ai " + str(age) + " de ani"


f("Rafael", 23)
```

Out[29]:
```
'Salut Rafael, ai 23 de ani'
```

Pentru tipuri complexe se pot folosi `list`, `dict`, `tuple` etc.

In [30]:
```
def salut_multi(nume: list[str], varste: list[int]) -> None:
    for n, v in zip(nume, varste):
        print(f"Salut {n}, ai {v} de ani")


salut_multi(["Ana", "Dan", "Maria"], [20, 21, 22])
```

```
Salut Ana, ai 20 de ani
Salut Dan, ai 21 de ani
Salut Maria, ai 22 de ani
```

Pentru adnotare de dicționare:

In [31]:
```
colegi: dict[str, list[str]] = {
    "Ana": ["0712121212", "026833333333"],
    "Dan": ["036811223344", "112"],
}
t: tuple[int, str] = (123, "reusite")
```

Se pot defini tipuri utilizator, folosindu-ne de cele disponibile:

In [32]:
```
type Point2D = tuple[float, float]


def plot_point(point: Point2D) -> bool:
    ## operatii
    return True


def rotate_point(p: Point2D, angle: float) -> Point2D:
    ## cod de rotatie a punctului p cu unghiul angle (in radiani)
    ## newPoint = ....
```

```
    # return ...
    raise NotImplementedError  # de implementat
```

Dacă o variabilă poate avea unul din mai multe tipuri posibile, putem urma exemplul:

In [33]:
```
def print_value(value: str | int | float) -> None:
    print(value)
```

Dacă o variabilă poate să fie de un anumit tip sau să vină cu valoarea `None`, se procedează precum mai jos:

In [34]:
```
def f(param: str | None) -> str:
    if param is not None:
        return param.upper()
    else:
        return ""
```

Pentru alte construcții: definirea de tipuri noi, callback functions, colecții generice etc. recomandăm consultarea bibliografiei.

Bibliografie recomandată

1. PEP484
2. typing — Support for type hints
3. Type hints cheat sheet (Python 3)

Module

Modulele sunt fișiere Python cu extensia .py, în care se găsesc implementări de funcții, clase, declarații de variabile. Importarea unui modul se face cu instrucțiunea `import`.

Exemplu: creăm un modul - fișierul Python mySmartModule.py - care conține o funcție ce calculează suma elementelor dintr-o listă:

```
# fisierul mySmartModule.py
def my_sum(lista):
    result = 0
    for item in lista:
        result += item
    return result
```

Utilizarea se face cu:

```
import mySmartModule

lista = [1, 2, 3]

suma = mySmartModule.my_sum(lista)
print(suma)
```

Se poate ca modulul să fie importat cu un nume mai scurt, sub forma:

```
import mySmartModule as msm
```

și în acest caz apelul se face cu:

```
suma = msm.my_sum(lista)
```

Putem afla ce pune la dispoziție un modul:

```
>>> dir(msm)
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
'__name__', '__package__', '__spec__', 'my_sum']
```

Elementele aflate între dublu underscore ('dunders') sunt adăugate automat de Python.

Dacă se dorește ca tot ceea ce e definit într-un modul să fie disponibil fără a mai face prefixare cu `nume_modul.nume_entitate`, atunci se poate proceda astfel:

```
from mySmartModule import *

print(my_sum([1, 2, 3]))
```

Se recomandă însă să se importe strict acele entități (funcții, tipuri) din modul care sunt utilizate; în felul acesta se evită suprascrierea prin import a altor entități deja importate:

```
from mySmartModule import my_sum

print(my_sum([1, 2, 3]))
```

Ordinea de căutare a modulelor este:

1. directorul curent
2. dacă nu se găsește modulul cerut, se caută în variabila de mediu `PYTHONPATH`, dacă e definită
3. dacă nu se găsește modulul cerut, se caută în calea implicită.

Calea de căutare implicită se găsește în variabila `path` din modulul sistem `sys`:

In [35]:
```
import sys

print(sys.path)
```

```
['C:\\Users\\Lucian\\AppData\\Local\\Programs\\Python\\Python313\\python313.zip',
'C:\\Users\\Lucian\\AppData\\Local\\Programs\\Python\\Python313\\DLLs', 'C:\\Users
\\Lucian\\AppData\\Local\\Programs\\Python\\Python313\\Lib', 'C:\\Users\\Lucian\\App
Data\\Local\\Programs\\Python\\Python313', '', 'C:\\Users\\Lucian\\AppData\\Local\\P
rograms\\Python\\Python313\\Lib\\site-packages', 'C:\\Users\\Lucian\\AppData\\Local
\\Programs\\Python\\Python313\\Lib\\site-packages\\win32', 'C:\\Users\\Lucian\\AppDa
ta\\Local\\Programs\\Python\\Python313\\Lib\\site-packages\\win32\\lib', 'C:\\Users
\\Lucian\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\site-packages\\pythonwi
n']
```

Dacă se dorește ca un modul scris de utilizator într-un director ce nu se găsește în lista de mai sus să fie accesibil pentru import, atunci calea către director trebuie adăugată la colecția `sys.path` :

In [36]:
```
sys.path.append("./my_modules/")
from mySmartModule import my_sum

print(my_sum([1, 2, 3]))
```

```
6
```

Un modul se poate folosi în două feluri:

1. pentru a pune la dispoziție diferite implementări de funcții sau de clase, sau variabile setate la anumite valori (de exemplu `math.pi`):

```
import math
print(math.pi)
```

2. se poate lansa de sine stătător, scriind în linia de comandă: `python mySmartModule.py` . Pentru acest caz, dacă se vrea să se execute o anumită secvență de cod, atunci se va scrie în modulul Python `mySmartModule.py` :

```
if __name__ == '__main__':
    #cod care se executa la lansarea directa a script-ului
```

Codul din secțiunea `if` scrisă ca mai sus nu se va executa când modulul este importat.

Exemplu:

```
def my_sum(lista):
    result = 0
    for item in lista:
        result += item
    return result

if __name__ == '__main__':
    print('Exemplu de utilizare')
    lista = list(range(100))
    print(my_sum(lista))
```

Pachete Python

Un pachet este o colecție de module. Din punct de vedere fizic, un pachet este o structură de directoare în care se găsesc module și alte pachete. Un director devine pachet atunci când conține un fișier `__init__.py` , care poate fi gol. Din Python 3.3 sunt acceptate și pachete fără acest fișier (*namespace packages*, PEP 420), dar pentru pachetele proprii se recomandă păstrarea lui.

Plecăm de la structura de directoare și fișiere:

```
---myUtils\
 |------ mySmartModule.py
 |------ __init__.py
```

Pentru importul funcției `my_sum` din fișierul `mySmartModule.py` aflat în directorul `myUtils` - care se dorește a fi pachet - s-ar scrie astfel:

```
from myUtils.mySmartModule import my_sum
print(my_sum([1, 2, 3]))
```

dar am prefera să putem scrie:

```
from myUtils import my_sum
print(my_sum([1, 2, 30]))
```

adică să nu mai referim modulul (fișierul) `mySmartModule` din cadrul pachetului `myUtils`. Pentru asta vom adăuga în fișierul `__init__.py` din directorul `myUtils` linia:

```
from .mySmartModule import my_sum
```

unde caracterul `.` de dinaintea numelui de modul `mySmartModule` se referă la calea relativă.

In [37]:
```
from myUtils import my_sum

print(my_sum([1, 2, 10, 300]))
```

```
313
```

În fișierul `__init__.py` se pun de regulă importurile care definesc ce oferă pachetul și, eventual, constante. Operațiile costisitoare, cum ar fi încărcarea de date de pe disc, se evită aici, pentru că se execută la fiecare `import`; ele se fac la cerere (lazy loading), în funcțiile care au nevoie de date.

Pentru cazul în care se dorește crearea de pachete destinate comunității și publicarea pe PyPI, se poate urma tutorialul oficial Packaging Python Projects. Acesta descrie configurarea proiectului prin fișierul `pyproject.toml`, construirea pachetului cu `python -m build` și publicarea cu `twine`, mai întâi pe serverul de test TestPyPI. Pentru cei care folosesc `uv`, același flux se reduce la comenzile `uv build` și `uv publish`: Building and publishing a package.

Alte exemple de utilizare de pachete sunt:

In [38]:
```
import re  # pachet pentru expresii regulate

my_string = "Am cumparat: mere, pere, prune... si caise"
tokens = re.split(r"\W+", my_string)
print(tokens)
```

```
['Am', 'cumparat', 'mere', 'pere', 'prune', 'si', 'caise']
```

In [39]:
```
import pickle
from pathlib import Path
```

```
favourite_colour = {"lion": "yellow", "kitty": "red"}
with open("save.pkl", "wb") as f:
    pickle.dump(favourite_colour, f)
del favourite_colour

with open("save.pkl", "rb") as f:
    favourite_colour_restored = pickle.load(f)
print("dupa deserializare:", favourite_colour_restored)

Path("save.pkl").unlink()  # sterge fisierul, indiferent de sistemul de operare
```

```
dupa deserializare: {'lion': 'yellow', 'kitty': 'red'}
```

Atenție: `pickle.load` execută cod în timpul încărcării. Un fișier pickle primit din altă sursă poate rula orice comandă pe calculatorul vostru. Se încarcă doar fișiere pickle create de voi sau de o sursă de încredere; pentru schimbul de date se folosesc formate care conțin doar date: CSV, JSON, `np.save` / `np.load`. Același risc există la `joblib.load` și `torch.load`, care folosesc intern pickle. Detalii: avertismentul din documentația oficială.

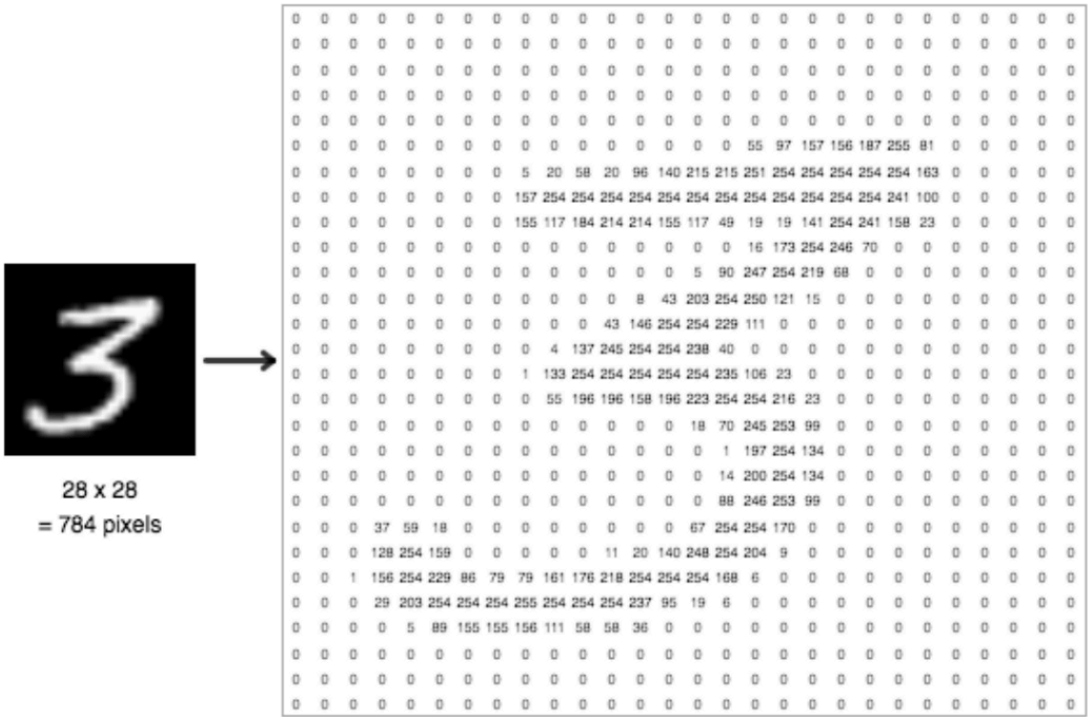
Pachetul NumPy

NumPy (Numerical Python) este pachetul de bază pentru calcule științifice în Python. Asigură suport pentru lucrul cu vectori și matrice multidimensionale, funcții dedicate precum sortare, operații din algebra liniară, procesare de semnal, calcule statistice de bază, generare de numere aleatoare etc. NumPy stă la baza multor altor pachete. Datele pe care le procesează trebuie să încapă în memoria RAM. NumPy are la bază cod C compilat și optimizat.

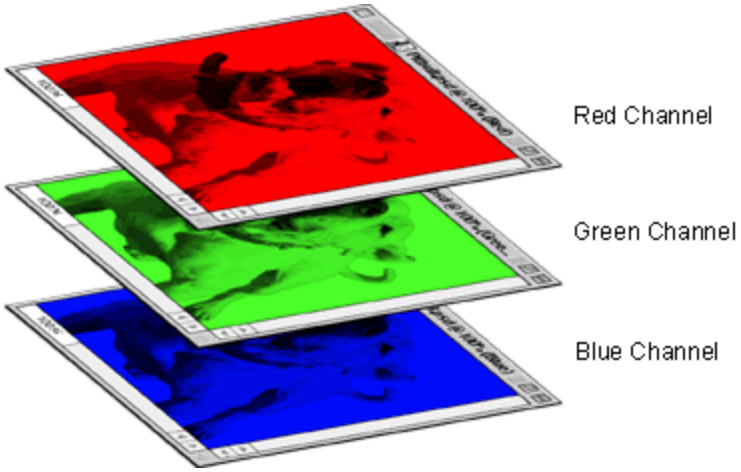
În destul de multe situații, datele sunt sau pot fi transformate în numere:

- o imagine în tonuri de gri poate fi văzută ca o matrice bidimensională de numere; fiecare număr reprezintă intensitatea pixelului (0 - negru, 255 - alb; frecvent se folosesc

valorile scalate în intervalul [0, 1])

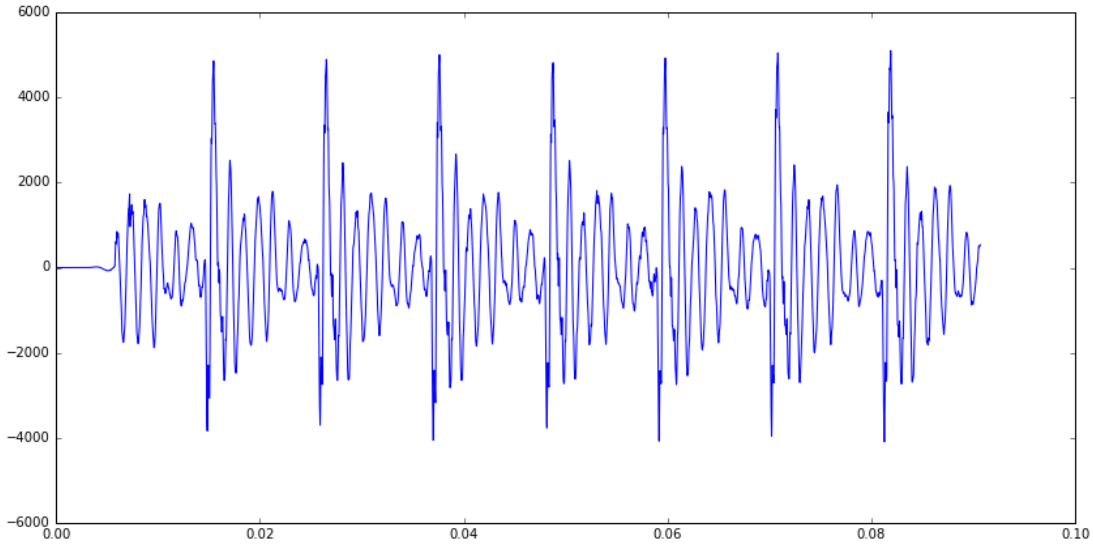


- o imagine color poate fi văzută ca o matrice cu trei dimensiuni: 3 matrice bidimensionale "paralele", corespunzătoare canalelor red, green, blue; pentru fiecare canal de culoare valorile pot fi între 0 și 255:



- un fișier audio este văzut ca unul/doi/k vectori unidimensionali, corespunzători cazurilor: mono, stereo, k canale. Valorile numerice în cazul unui fișier wav reprezintă

deplasarea membranei microfonului, discretizată în timp și amplitudine:



- un text poate fi tradus în vectori numerici prin tehnici precum Bag of words sau Word2vec.

Reprezentarea este mult mai eficientă decât pentru listele Python; codul scris cu NumPy apelează biblioteci compilate în cod nativ. Dacă se scrie cod vectorizat, eficiența rulării e și mai mare.

Tipul cel mai comun din NumPy este `ndarray` - n-dimensional array.

In [40]:
```
# import de pachet; traditional se foloseste abrevierea np pentru numpy
import numpy as np

# crearea unui vector pornind de la o lista Python
x = np.array([1, 4, 2, 5, 3])

# tipul variabilei x; se observa ca e tip numpy
print(type(x))
# toate elementele din array sunt de acelasi tip
print(x.dtype)

# specificarea explicita a tipului de reprezentare a datelor in array
y = np.array([1, 2, 3], dtype=np.float16)
print(y.dtype)
```

```
<class 'numpy.ndarray'>
int32
float16
```

In [41]:
```
# cazuri frecvent folosite
all_zeros = np.zeros(10, dtype=np.int64)
print(all_zeros)
# tiparire nr de elemente pe fiecare dimensiune
print(all_zeros.shape)
```

```
[0 0 0 0 0 0 0 0 0 0]
(10,)
```

In [42]:
```
# matrice 2d
mat = np.array([[1, 2, 3], [4, 5, 6]])
print(mat)
print(mat.shape)
print(mat[0, 1])
```

```
[[1 2 3]
 [4 5 6]]
(2, 3)
2
```

In [43]:
```
# matrice de valori constante:
mat_7 = np.full(shape=(3, 10), fill_value=7)
print(mat_7)

mat_7_v2 = np.ones((3, 10)) * 7
print(mat_7_v2)
```

```
[[7 7 7 7 7 7 7 7 7 7]
 [7 7 7 7 7 7 7 7 7 7]
 [7 7 7 7 7 7 7 7 7 7]]
[[7. 7. 7. 7. 7. 7. 7. 7. 7. 7.]
 [7. 7. 7. 7. 7. 7. 7. 7. 7. 7.]
 [7. 7. 7. 7. 7. 7. 7. 7. 7. 7.]]
```

Numărul de dimensiuni se determină cu:

In [44]:
```
print("Numarul de dimensiuni pentru vectorul all_zero:", all_zeros.ndim)
print("Numarul de dimensiuni pentru matricea mat:", mat.ndim)
```

```
Numarul de dimensiuni pentru vectorul all_zero: 1
Numarul de dimensiuni pentru matricea mat: 2
```

iar numărul total de elemente, respectiv dimensiunea în octeți a unui element oarecare (un ndarray are elemente de același tip, întotdeauna):

In [45]:
```
print(
    f"mat size: {mat.size}\nmat element size: {mat.itemsize} bytes\nmat.dtype:{mat.
)
```

```
mat size: 6
mat element size: 4 bytes
mat.dtype:int32
```

In [46]:
```
# cazuri comune
all_ones = np.ones((3, 5))
print(all_ones)
print(np.eye(3))
```

```
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [47]:
```
# valori echidistante intr-un interval; capetele intervalului fac parte din valoril
print(np.linspace(0, 10, 5))
```

```
[ 0.   2.5  5.   7.5 10. ]
```

In [48]:
```
# similar cu functia range din Python: se genereaza de la primul parametru, cu pasu
# ultima valoare generata fiind strict mai mica decat al doilea parametru
vector_de_valori = np.arange(0, 10, 3)
print(vector_de_valori)
print(type(vector_de_valori))
```

```
[0 3 6 9]
<class 'numpy.ndarray'>
```

In [49]:
```
# numere aleatoare; folosim rng pentru a avea un generator de numere aleatoare care
rng = np.random.default_rng(42)
x = rng.random((2, 3))
print(x)
```

```
[[0.77395605 0.43887844 0.85859792]
 [0.69736803 0.09417735 0.97562235]]
```

Tipurile de date folosibile pentru ndarrays (NumPy 2.x) sunt:

Tip	Explicație
`bool` / `bool_`	Valoare logică (`True` sau `False`), stocată pe un octet
`int8`	Întreg pe 8 biți (-128 ... 127)
`int16`	Întreg pe 16 biți (-32768 ... 32767)
`int32`	Întreg pe 32 de biți (-2147483648 ... 2147483647)
`int64`	Întreg pe 64 de biți (-9223372036854775808 ... 9223372036854775807)
`uint8`	Întreg fără semn pe 8 biți (0 ... 255); tipul uzual pentru pixelii unei imagini
`uint16`	Întreg fără semn pe 16 biți (0 ... 65535)
`uint32`	Întreg fără semn pe 32 de biți (0 ... 4294967295)
`uint64`	Întreg fără semn pe 64 de biți (0 ... 18446744073709551615)
`int_`	Tipul întreg implicit; în NumPy 2 este identic cu `intp` , adică `int64` pe sisteme pe 64 de biți, inclusiv Windows
`intp` / `uintp`	Întreg de dimensiunea unui pointer, folosit pentru indexare (`int64` / `uint64` pe 64 de biți)
`intc`	Identic cu `int` din C (de regulă `int32`)
`long` / `ulong`	Identic cu `long` din C: `int32` pe Windows, `int64` pe Linux și macOS
`float16`	Virgulă mobilă, precizie înjumătățită: 1 bit semn, 5 biți exponent, 10 biți mantisă (~3 cifre zecimale semnificative)
`float32`	Virgulă mobilă, precizie simplă: 1 bit semn, 8 biți exponent, 23 biți mantisă (~7 cifre); tipul uzual pentru rețele neuronale

Tip	Explicație
`float64` / `double`	Virgulă mobilă, precizie dublă: 1 bit semn, 11 biți exponent, 52 biți mantisă (~15-16 cifre); tipul implicit pentru numere reale
`complex64`	Număr complex, reprezentat prin două valori `float32` (partea reală și partea imaginară)
`complex128`	Număr complex, reprezentat prin două valori `float64`; tipul implicit pentru numere complexe
`str_`	Șir de caractere Unicode de lungime fixă (de exemplu `<U5`: maximum 5 caractere)
`StringDType()`	Șir de caractere de lungime variabilă, nou în NumPy 2.0 (`np.dtypes.StringDType()`)

Dacă nu se precizează tipul, `np.array([1, 2])` produce `int64`, iar `np.array([1.0, 2.0])` produce `float64`.

O operație utilă este schimbarea formei unui array:

In [50]:
```
# dintr-un vector intr-o matrice
vec = np.arange(10)
mat = vec.reshape(2, 5)
print(vec)
print(mat)
```

```
[0 1 2 3 4 5 6 7 8 9]
[[0 1 2 3 4]
 [5 6 7 8 9]]
```

In [51]:
```
# ... si invers:
vec2 = mat.flatten()
print(vec2)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

Tablourile pot fi concatenate, specificându-se axa:

In [52]:
```
a = np.array([[1, 2], [3, 4]], float)
b = np.array([[5, 6], [7, 8]], float)
```

In [53]:
```
# concatenare pe verticala
stiva_verticala = np.concatenate((a, b), axis=0)
print(stiva_verticala)
```

```
[[1. 2.]
 [3. 4.]
 [5. 6.]
 [7. 8.]]
```

Conceptul de axă se definește pentru tablourile cu mai mult de o dimensiune. Pentru un tablou cu două dimensiuni, axa 0 parcurge pe verticală tabloul, axa 1 parcurge pe orizontală. Unele funcții iau în considerare axa de lucru:

In [54]:
```
# concatenare pe orizontala
stiva_orizontala = np.concatenate((a, b), axis=1)
print(stiva_orizontala)
```

```
[[1. 2. 5. 6.]
 [3. 4. 7. 8.]]
```

In [55]:
```
# echivalent cu:
stiva_verticala = np.vstack((a, b))
stiva_orizontala = np.hstack((a, b))
print(stiva_verticala)
print(stiva_orizontala)
```

```
[[1. 2.]
 [3. 4.]
 [5. 6.]
 [7. 8.]]
[[1. 2. 5. 6.]
 [3. 4. 7. 8.]]
```

In [56]:
```
matrice = np.arange(15).reshape(3, 5)
print(matrice)
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]
```

In [57]:
```
suma_pe_coloane = np.sum(matrice, axis=0)
print(suma_pe_coloane)
```

```
[15 18 21 24 27]
```

In [58]:
```
suma_pe_linii = np.sum(matrice, axis=1)
print(suma_pe_linii)
```

```
[10 35 60]
```

Operații cu ndarrays

Sunt implementate operațiile matematice uzuale din algebra liniară: înmulțire cu scalari, adunare, scădere, înmulțire de matrice.

Exemple:

- adunare
- produs Hadamard, produs matricial, produs scalar:

In [59]:
```
# inmultire de matrice cu scalar
a = np.array([[1, 2, 3], [4, 5, 6]])
print("a=\n", a)
```

```
b = a * 10
print("b=\n", b)
```

```
a=
 [[1 2 3]
 [4 5 6]]
b=
 [[10 20 30]
 [40 50 60]]
```

In [60]:
```
# adunare, scadere: +, -
suma = a + b
print(suma)
diferenta = a - b
print(diferenta)
```

```
[[11 22 33]
 [44 55 66]]
[[ -9 -18 -27]
 [-36 -45 -54]]
```

Operatorul de înmulțire * este implementat altfel decât în algebra liniară: pentru două matrice cu aceleași dimensiuni se face înmulțirea elementelor aflate pe poziții identice, adică: `c[i, j] = a[i, j] * b[i, j]`. Este așa-numitul produs Hadamard, frecvent întâlnit în machine learning.

In [61]:
```
# inmultirea folosind * duce la inmultire element cu element (produs Hadamard): c[i
c = a * b
print(c)
for i in range(c.shape[0]):  # c.shape[0] = numarul de linii ale matricei c
    for j in range(c.shape[1]):  # c.shape[1] = numarul de coloane ale matricei c
        print(c[i, j] == a[i, j] * b[i, j])
```

```
[[ 10  40  90]
 [160 250 360]]
True
True
True
True
True
True
```

Operațiile folosesc biblioteci de algebră liniară, optimizate pentru microprocesoarele actuale. Se recomandă folosirea acestor implementări în loc de a face operațiile manual cu ciclări `for`:

In [62]:
```
# creare de matrice
matrix_shape = (100, 100)
a_big = rng.random(matrix_shape)
b_big = rng.random(matrix_shape)
```

In [63]:
```
%%timeit
c_big = np.empty_like(a_big)
for i in range(c_big.shape[0]):
```

```
    for j in range(c_big.shape[1]):
        c_big[i, j] = a_big[i, j] * b_big[i, j]
```

```
3.97 ms ± 401 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

In [64]:
```
%%timeit
c_big = a_big * b_big
```

```
4.73 µs ± 310 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

In [65]:
```
# 'ridicarea la putere' folosind ** : fiecare element al matricei este ridicat la p
print("matricea initiala:\n", a)
putere = a**2
print("dupa ridicarea la puterea 2:\n", putere)
putere_3 = np.power(a, 3)
print("dupa ridicarea la puterea 3:\n", putere_3)
```

```
matricea initiala:
 [[1 2 3]
 [4 5 6]]
dupa ridicarea la puterea 2:
 [[ 1  4  9]
 [16 25 36]]
dupa ridicarea la puterea 3:
 [[  1   8  27]
 [ 64 125 216]]
```

Se poate folosi operatorul / pentru a face împărțirea punctuală (element cu element) a valorilor din două matrice:

In [66]:
```
print("a=", a)
print("b=", b)
print("a/b=", a / b)
```

```
a= [[1 2 3]
 [4 5 6]]
b= [[10 20 30]
 [40 50 60]]
a/b= [[0.1 0.1 0.1]
 [0.1 0.1 0.1]]
```

In [67]:
```
# ridicarea la putere a unei matrice patratice, asa cum e definita in algebra linia
patratica = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
ridicare_la_putere = np.linalg.matrix_power(patratica, 3)
print(ridicare_la_putere)
```

```
[[ 468  576  684]
 [1062 1305 1548]
 [1656 2034 2412]]
```

Dacă se apelează o funcție matematică definită în NumPy pe un ndarray, rezultatul va fi tot un ndarray de aceeași formă ca și intrarea, dar cu elementele calculate prin aplicarea funcției respective:

In [68]:
```
x = np.arange(6).reshape(2, 3)
print(x)
```

```
y = np.exp(x)
assert x.shape == y.shape
for i in range(0, x.shape[0]):
    for j in range(0, x.shape[1]):
        assert np.allclose(
            y[i, j], np.exp(x[i, j])
        )  # folosim np.allclose pentru a verifica egalitatea cu o toleranta, a.i.
```

```
[[0 1 2]
 [3 4 5]]
```

In [69]:
```
# produs algebric de matrice:
a = rng.random((3, 5))
b = rng.random((5, 10))
assert a.shape[1] == b.shape[0]
c = np.dot(a, b)
# se poate scrie echivalent
c = a.dot(b)
assert a.shape[0] == c.shape[0] and b.shape[1] == c.shape[1]
# sau folosind operatorul @
c = a @ b
```

NumPy definește o serie de funcții ce pot fi utilizate: `all, any, apply_along_axis, argmax, argmin, argsort, average, bincount, ceil, clip, conj, corrcoef, cov, cross, cumprod, cumsum, diff, dot, floor, inner, lexsort, max, maximum, mean, median, min, minimum, nonzero, outer, prod, real, round, sort, std, sum, trace, transpose, var, vdot, vectorize, where` - documentate aici.

Indexare

Până acum, pentru referirea elementelor de la anumite poziții s-au folosit indici simpli de forma:

```
vector[indice]
# sau
matrice[i, j]
```

In [70]:
```
vector = np.arange(10)
print(vector)
print(f"vector[4]={vector[4]}")
```

```
[0 1 2 3 4 5 6 7 8 9]
vector[4]=4
```

In [71]:
```
matrice = np.arange(12).reshape(3, 4)
print(matrice)
print(matrice[2, 1])
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
9
```

Avem posibilitatea de a face referire a mai multor elemente dintr-un tablou.

Slicing

Prin `slicing` există posibilitatea de a face referire la un întreg subset de elemente, de exemplu pentru vectori:

In [72]:
```
vector = 10 * np.arange(10)
print(vector)
print(
    vector[2:6]
)  # remarcam ca indicele din dreapta este cu rol de "exclusiv", nu contribuie la r
```

```
[ 0 10 20 30 40 50 60 70 80 90]
[20 30 40 50]
```

In [73]:
```
# slicing cu pas: start:stop:pas
vector[2:8:2]
```

Out[73]:
```
array([20, 40, 60])
```

Pentru matrice putem folosi:

In [74]:
```
matrice = 10 * np.arange(20).reshape(4, 5)
print(matrice)
```

```
[[  0  10  20  30  40]
 [ 50  60  70  80  90]
 [100 110 120 130 140]
 [150 160 170 180 190]]
```

In [75]:
```
print(matrice[1,])
# care e totuna cu forma mai explicita:
print(matrice[1, :])
# sau cu forma:
print(matrice[1, ...])
```

```
[50 60 70 80 90]
[50 60 70 80 90]
[50 60 70 80 90]
```

In [76]:
```
# putem selecta domenii de indici, pe fiecare axa
matrice[1:3, :]
```

Out[76]:
```
array([[ 50,  60,  70,  80,  90],
       [100, 110, 120, 130, 140]])
```

In [77]:
```
# indexare pe fiecare dimensiune
matrice[1:3, 2:4]
```

Out[77]:
```
array([[ 70,  80],
       [120, 130]])
```

Indexarea cu liste de indici (fancy indexing)

În locul unui indice sau al unui domeniu `start:stop:pas` se poate da o listă (sau un tablou) de indici. Rezultatul conține elementele de pe pozițiile cerute, în ordinea din listă; un indice poate apărea de mai multe ori:

In [78]:
```
indici = [1, 3, 2, 7, 7]
print(vector)
print(vector[indici])
```

```
[ 0 10 20 30 40 50 60 70 80 90]
[10 30 20 70 70]
```

Spre deosebire de slicing, care returnează un view, indexarea cu liste de indici returnează întotdeauna o copie, chiar dacă elementele selectate sunt aceleași:

In [79]:
```
felie = vector[2:6]  # slicing -> view
selectie = vector[[2, 3, 4, 5]]  # lista de indici -> copie
print(felie, selectie)
print(np.shares_memory(vector, felie), np.shares_memory(vector, selectie))
```

```
[20 30 40 50] [20 30 40 50]
True False
```

Pentru matrice, o listă de indici pe o axă selectează linii sau coloane întregi. Dacă se dau liste pe ambele axe, ele se împerechează element cu element: `matrice[[0, 2], [1, 3]]` înseamnă elementele `[0, 1]` și `[2, 3]`, nu blocul format din liniile 0, 2 și coloanele 1, 3. Pentru bloc se folosește `np.ix_`:

In [80]:
```
print(matrice[[0, 2]])  # liniile 0 si 2
print(matrice[:, [1, 3]])  # coloanele 1 si 3
print(matrice[[0, 2], [1, 3]])  # elementele [0, 1] si [2, 3]
print(matrice[np.ix_([0, 2], [1, 3])])  # blocul: liniile 0, 2 si coloanele 1, 3
```

```
[[  0  10  20  30  40]
 [100 110 120 130 140]]
[[ 10  30]
 [ 60  80]
 [110 130]
 [160 180]]
[ 10 130]
[[ 10  30]
 [110 130]]
```

În machine learning, indexarea cu liste de indici apare la amestecarea și împărțirea datelor: se generează o permutare a indicilor liniilor și cu ea se aleg exemplele de antrenare și de test.

In [81]:
```
X = np.arange(20).reshape(10, 2)  # 10 exemple, 2 caracteristici
idx = rng.permutation(len(X))  # indicii liniilor, amestecati
train, test = X[idx[:8]], X[idx[8:]]
print(idx)
print(test)
```

```
[3 0 8 2 4 5 7 6 9 1]
[[18 19]
 [ 2  3]]
```

Indexarea logică

Asupra elementelor unui tablou se pot aplica operații logice; obținem un tablou de aceeași formă ca și tabloul inițial, dar plin cu valori `True` și `False` în funcție de rezultatul aplicării operației logice:

In [82]:
```
a = np.array([[1, 2], [3, 4], [5, 6]])
print(a)
print(a > 2)
```

```
[[1 2]
 [3 4]
 [5 6]]
[[False False]
 [ True  True]
 [ True  True]]
```

Tabloul rezultat în urma aplicării operației logice poate fi folosit pentru indexare. Se vor returna doar acele elemente care satisfac condiția logică cerută:

In [83]:
```
mai_mare_ca_2 = a > 2
print(a[mai_mare_ca_2])
# direct
print(a[a > 2])
```

```
[3 4 5 6]
[3 4 5 6]
```

Dacă se doresc expresii mai complicate, de exemplu elemente care sunt mai mari ca 2 și mai mici ca 6, atunci scriem:

In [84]:
```
a[np.logical_and(a > 2, a < 6)]
```

Out[84]:
```
array([3, 4, 5])
```

Mai există: `np.logical_or`, `np.logical_not`, `np.logical_xor`.

O expresie logică utilă este următoarea: se cere obținerea doar a acelor elemente care sunt definite - adică nu sunt NaN:

In [85]:
```
tab = np.array([[1.0, 2.3, np.nan, 4], [10, np.nan, np.nan, 0]])
print(tab[~np.isnan(tab)])
```

```
[ 1.   2.3  4.  10.   0. ]
```

Indexarea logică permite specificarea elementelor dintr-un tablou pentru care se efectuează anumite operații:

In [86]:
```
# numerele pare se inmultesc cu 10, celelalte raman cum sunt
tablou = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print("Inainte de modificare:\n", tablou)
tablou[tablou % 2 == 0] *= 10
print("Dupa modificare:\n", tablou)
```

```
Inainte de modificare:
 [[1 2 3 4]
 [5 6 7 8]]
Dupa modificare:
 [[ 1 20  3 40]
 [ 5 60  7 80]]
```

Modificarea se poate face și doar pe o anumită axă:

In [87]:
```
tablou = np.array([[1, 2, 3, 4], [5, 6, 7, 8]], dtype=np.float32)
print("Inainte de modificare\n", tablou)
# coloanele 0, 2, 3 se modifica
bool_columns = [True, False, True, True]
tablou[:, bool_columns] = (tablou[:, bool_columns] + 3) / 10
print("Dupa modificare\n", tablou)
```

```
Inainte de modificare
 [[1. 2. 3. 4.]
 [5. 6. 7. 8.]]
Dupa modificare
 [[0.4 2.  0.6 0.7]
 [0.8 6.  1.  1.1]]
```

Diferente dpdv al memoriei intre slicing si indexare

Slicing-ul returnează un 'view' al tabloului, peste care se pot aplica modificări ale conținutului originar:

In [88]:
```
print("Inainte:\n", tab)

linie = tab[1, :]  # slicing -> view: aceleasi date ca tab
linie[:] = -100
print("Dupa modificarea view-ului:\n", tab)  # tab s-a modificat
```

```
Inainte:
 [[ 1.   2.3  nan  4. ]
 [10.   nan  nan  0. ]]
Dupa modificarea view-ului:
 [[   1.     2.3    nan    4. ]
 [-100.  -100.  -100.  -100. ]]
```

Operarea pe view înseamnă modificarea valorilor corespunzătoare din tabloul inițial, deoarece view și tabloul partajează memorie. Indexarea logică și cea cu liste de indici, însă, produc o copie; modificarea elementelor din copie nu afectează zona de memorie de proveniență:

In [89]:
```
linie = tab[1, :]  # slicing -> view: aceleasi date ca tab

print("Demo indexare logica: tab=\n", tab)

valori = tab[~np.isnan(tab)]  # indexarea logica produce copie
valori[:] = 0
print("Dupa modificarea copiei: tab=\n", tab)  # tab NU s-a modificat

print(f"{np.shares_memory(tab, linie)=}\n{np.shares_memory(tab, valori)=}")
```

```
Demo indexare logica: tab=
 [[   1.     2.3    nan    4. ]
 [-100.  -100.  -100.  -100. ]]
Dupa modificarea copiei: tab=
 [[   1.     2.3    nan    4. ]
 [-100.  -100.  -100.  -100. ]]
np.shares_memory(tab, linie)=True
np.shares_memory(tab, valori)=False
```

Bibliografie recomandată

1. NumPy Glossary
2. NumPy: Indexing on ndarrays
3. Scientific Python Lectures - NumPy: indexing and slicing
4. M. Scott Shell, *An introduction to Numpy and Scipy*

Broadcasting

Broadcasting este un mecanism prin care se permite - în anumite circumstanțe - operarea cu matrice de dimensiuni ce nu sunt compatibile din punct de vedere al dimensiunii. De exemplu, urmând strict definiția matematică a adunării, matricele `a` și `b` de mai jos nu se pot aduna:

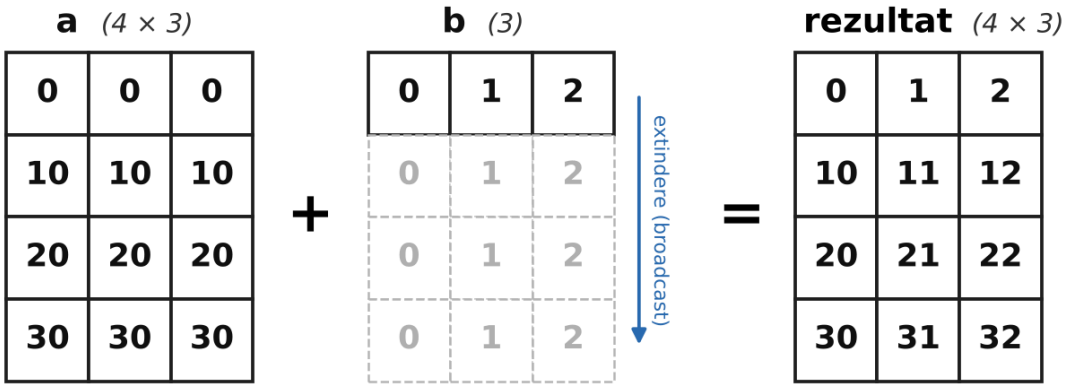
In [90]:
```
a = np.array(
    [[0.0, 0.0, 0.0], [10.0, 10.0, 10.0], [20.0, 20.0, 20.0], [30.0, 30.0, 30.0]]
)
b = np.array([0.0, 1.0, 2.0])

print(f"a=\n{a}\n")
print(f"b=\n{b}\n")
```

```
a=
[[ 0.  0.  0.]
 [10. 10. 10.]
 [20. 20. 20.]
 [30. 30. 30.]]

b=
[0. 1. 2.]
```

Prin broadcasting, `b` se comportă ca și cum linia ar fi repetată pentru fiecare linie a lui `a`. NumPy nu copiază efectiv datele; repetarea este doar virtuală, ceea ce face ca operația să fie rapidă și economică în memorie.



Vectorul b este replicat pe fiecare linie a lui a (rândurile punctate sunt virtuale, nu ocupă memorie).

In [91]:
```
# broadcasting
result = a + b
print(f"result=\n{result}\n")
```

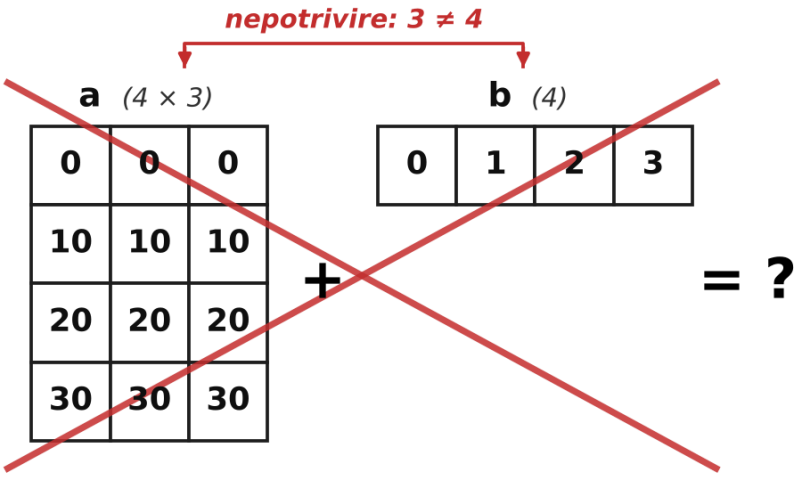
```
result=
[[ 0.  1.  2.]
 [10. 11. 12.]
 [20. 21. 22.]
 [30. 31. 32.]]
```

Când se operează cu două tablouri, NumPy compară dimensiunile - atributul `shape` - element cu element, începând cu ultima dimensiune. Două dimensiuni sunt compatibile când:

1. sunt egale, sau
2. una din ele este 1

Dacă tablourile au număr diferit de dimensiuni, forma celui cu mai puține dimensiuni se completează cu 1 la stânga. De exemplu, `b` cu forma `(3,)` este tratat ca având forma `(1, 3)`, deci se poate aduna cu `a`, de formă `(4, 3)`.

Regulile de mai sus nu sunt îndeplinite, de exemplu, pentru:



Eroare: ultimele dimensiuni (3 și 4) nu sunt egale și niciuna nu este 1, deci broadcasting-ul nu este posibil.

sau pentru cazul simplu de mai jos:

In [92]:
```
x = np.arange(4)
y = np.ones(5)
print(x.shape, y.shape)
# print(x+y) # ValueError: operands could not be broadcast together with shapes (4,
```

```
(4,) (5,)
```

Exemplu concret

(Sursa) Se dau porțiile de grăsimi, proteine și carbohidrați dintr-un meniu. Să se calculeze câte calorii reprezintă. Numărul de calorii se determină astfel:

1. nr. de calorii pentru grăsimi = 9 * grame grăsimi
2. nr. de calorii pentru proteine = 4 * grame proteine
3. nr. de calorii pentru carbohidrați = 4 * grame carbohidrați

Aliment (1 porție)	Grăsimi (g)	Proteine (g)	Carbohidrați (g)
Broccoli	0,3	2,5	3,5
Piept de pui	2,9	27,5	0
Banană	0,4	1,3	23,9
Migdale crude	14,4	6	2,3
...	...	...	...

grame: n × 3

× *element cu element*

[9 4 4]

kcal per gram

Aliment (1 porție)	Grăsimi (kcal)	Proteine (kcal)	Carbohidrați (kcal)
Broccoli	2,7	10	14
Piept de pui	26,1	110	0
Banană	3,6	5,2	95,6
Migdale crude	129,6	24	9,2
...	...	...	...

calorii: n × 3

Vectorul [9, 4, 4] (formă 3) este extins pe fiecare rând: 1 g grăsimi = 9 kcal, 1 g proteine = 4 kcal, 1 g carbohidrați = 4 kcal

Rezolvarea se face prin aplicarea de broadcasting:

In [93]:
```
weights = np.array([[0.3, 2.5, 3.5], [2.9, 27.5, 0], [0.4, 1.3, 23.9], [14.4, 6, 2.

cal_per_g = np.array([9, 4, 4])
```

```
# broadcasting
calories = weights * cal_per_g

print("Calorii:\n", calories)
```

```
Calorii:
 [[  2.7  10.   14. ]
 [ 26.1 110.    0. ]
 [  3.6   5.2  95.6]
 [129.6  24.    9.2]]
```

Un alt exemplu este:

In [94]:
```
x = np.arange(4).reshape(4, 1)
print("x shape: ", x.shape)
print("x:\n", x)
```

```
x shape:  (4, 1)
x:
 [[0]
 [1]
 [2]
 [3]]
```

In [95]:
```
y = np.arange(5).reshape(1, 5)
print("y shape: ", y.shape)
print("y:\n", y)
```

```
y shape:  (1, 5)
y:
 [[0 1 2 3 4]]
```

In [96]:
```
z = x + y
print("z shape:", z.shape)
print("z\n", z)
```

```
z shape: (4, 5)
z
 [[0 1 2 3 4]
 [1 2 3 4 5]
 [2 3 4 5 6]
 [3 4 5 6 7]]
```

In [97]:
```
# clonam x si cream coloane identice
x_broadcast = np.tile(x, (1, 5))
x_broadcast
```

Out[97]:
```
array([[0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [2, 2, 2, 2, 2],
       [3, 3, 3, 3, 3]])
```

In [98]:
```
# clonam y si cream linii identice:
y_broadcast = np.tile(y, (4, 1))
y_broadcast
```

Out[98]:
```
array([[0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4],
       [0, 1, 2, 3, 4]])
```

In [99]:
```
z_broadcast = x_broadcast + y_broadcast
assert np.all(z_broadcast == z)
```

Bibliografie

1. Broadcasting
2. CS231n - Numpy broadcasting

Vectorizare

Vectorizarea înseamnă exprimarea unei prelucrări asupra unor colecții de date ca operație pe tablouri întregi, în locul buclelor explicite scrise în Python care parcurg elementele unul câte unul. Bucla/bucclele se mută din interpretorul Python în cod compilat (C) din interiorul bibliotecii NumPy.

Exemplu

Se dau două colecții de numere: prima conține distanțe parcurse, a doua timpul necesar pentru parcurgere. Se cere determinarea vitezelor corespunzătoare. Se va face implementare folosind ciclare (clasic) și vectorizare.

In [100…
```
distante = [10, 20, 23, 14, 33, 45]
timpi = [0.3, 0.44, 0.9, 1.2, 0.7, 1.1]
```

In [101…
```
# var 1: folosind ciclare
viteze = []
for i in range(len(distante)):
    viteze.append(distante[i] / timpi[i])

print("Viteze: ", viteze)
```

```
Viteze:  [33.333333333333336, 45.45454545454545, 25.555555555555554, 11.666666666666
668, 47.142857142857146, 40.90909090909091]
```

In [102…
```
# var 2: vectorizare
# vectorizarea numpy lucreaza peste tablouri, primul pas este obtinerea de tablouri

distante_array = np.array(distante)
timpi_array = np.array(timpi)

# se folosesc operatii NumPy care trateaza tablourile in intregime. Codul C folosit
# foloseste facilitatile de executie Single Instruction Multiple Data (SIMD) din mi
viteze_array = distante_array / timpi_array
```

```
print(viteze_array)
```

```
[33.33333333 45.45454545 25.55555556 11.66666667 47.14285714 40.90909091]
```

Beneficii

1. Execuție rapidă
2. Cod mai scurt și deseori mai clar

Exemplu: să se calculeze:

$$\sum_{i=0}^{N-1} (i\%3 - 1) \cdot i$$

In [103…
```
# functie Python implementata naiv

N = 100000


def func_python(N):
    d = 0.0
    for i in range(N):
        d += (i % 3 - 1) * i
    return d


print(func_python(N))
```

```
-33333.0
```

In [104…
```
%timeit func_python(N)
```

```
8.3 ms ± 241 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

In [105…
```
# functie rescrisa folosind facilitatile NumPy si vectorizare


def func_numpy(N):
    i_array = np.arange(N)
    return ((i_array % 3 - 1) * i_array).sum()


print(func_numpy(N))
```

```
-33333
```

In [106…
```
%timeit func_numpy(N)
```

```
408 µs ± 17.4 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

Majoritatea operatorilor și a funcțiilor NumPy lucrează element cu element (sunt numite și universal functions, sau ufuncs) și într-un mod optimizat (SIMD). Următoarele sunt ufuncs:

- operatori aritmetici: `+ - * / // % **`
- operații pe biți: `& | ~ ^ >> <<`
- comparații: `< <= > >= == !=`
- funcții matematice: `np.sin` , `np.log` , `np.exp` , ...
- funcții speciale: `scipy.special.*`

Deși unele funcții din NumPy se regăsesc și în Python (ex: `sum` , `min` , `max`), folosirea de ufunc duce la execuție mai rapidă:

In [107…
```
c = rng.random(N).tolist()
```

In [108…
```
%timeit sum(c)
```

```
614 µs ± 31.4 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

In [109…
```
# vectorizare
c_array = np.array(c)
```

In [110…
```
%timeit c_array.sum()
```

```
32.7 µs ± 165 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

Compilare just-in-time cu Numba

O parte considerabilă din codul Python poate beneficia de compilarea just-in-time (JIT) oferită de Numba. Pentru aceasta este nevoie ca funcția pentru care se vrea execuție rapidă să fie decorată cu `@jit` ; ca urmare, funcția va fi tradusă în cod mașină, mult mai rapid decât codul interpretat.

Exemplul de mai jos se folosește de aceeași implementare ca mai sus, doar cu folosirea de decorator. Pentru mai multe detalii recomandăm parcurgerea materialului Numba. Un decorator este o funcție care primește o altă funcție și întoarce o variantă modificată a ei; se aplică scriind `@nume_decorator` deasupra definiției. `@jit` întoarce o variantă compilată a funcției `func_python_numba` .

In [111…
```
from numba import jit

N = 100000

@jit
def func_python_numba(N):
    d = 0.0
    for i in range(N):
        d += (i%3-1) * i
    return d

%timeit func_python_numba(N)
```

```
90 µs ± 1.12 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Se observă timpul de execuție semnificativ mai redus față de variantele Python și cod vectorizat. Există însă limitări ale pachetului Numba, nu orice funcție beneficiază de compilarea JIT.

In [112…
```
@jit
def func_numpy_jit(N):
    i_array = np.arange(N)
    return ((i_array % 3 - 1 ) * i_array).sum()

%timeit func_numpy_jit(N)
```

```
229 µs ± 110 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Exercițiu

Se dau n puncte în spațiul bidimensional, prin coordonatele lor memorate în 2 vectori **x** și **y**. Să se determine care este cea mai apropiată pereche de puncte, considerând distanța euclidiană:

$$d^2((x_i, y_i), (x_j, y_j)) = (x_i - x_j)^2 + (y_i - y_j)^2$$

In [113…
```
n = 1000
x = rng.random(size=n)
y = rng.random(size=n)
```

Varianta 1: se calculează matricea pătratelor distanțelor de dimensiune n*n. d[i, j] va reprezenta pătratul distanței între punctul de coordonate (xi, yi) și cel de coordonate (xj, yj).

In [114…
```
%%timeit

d = np.empty((n, n))
for i in range(n):
    for j in range(n):
        d[i, j] = (x[i] - x[j])**2 + (y[i]-y[j])**2
```

```
740 ms ± 15.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

In [115…
```
# calculul perechii de puncte i, j cu i != j pentru care distanta e minima
def pereche_apropiata(mat):
    # se face o copie a matricei pentru a nu modifica diagonala originalului
    mat = mat.copy()
    n = mat.shape[0]
    # distanta dintre un punct si el insusi este intotdeauna 0; se vor exclude aces
    i = np.arange(n)
    mat[i, i] = np.inf
    pos_flatten = np.argmin(mat)
    return pos_flatten // n, pos_flatten % n


# print(pereche_apropiata(d))
```

Varianta 2: broadcasting, vectorizare pentru calculul matricei d

`x[:, np.newaxis]` transformă vectorul de formă `(n,)` într-o matrice coloană, cu forma `(n, 1)`, iar `x[np.newaxis, :]` îl transformă într-o matrice linie, adică de forma `(1, n)`. Prin broadcasting, diferența lor are forma `(n, n)`: elementul `[i, j]` este `x[i] - x[j]`.

In [116…
```
%%timeit

dx = (x[:, np.newaxis] - x[np.newaxis, :]) ** 2
dy = (y[:, np.newaxis] - y[np.newaxis, :]) ** 2

d = dx + dy
```

```
10.9 ms ± 265 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Pentru afișarea perechii de puncte, comentați `%%timeit` din celula anterioară, rulați-o din nou, apoi rulați `print(pereche_apropiata(d))` într-o celulă de cod.

In [117…
```
# varianta cu numba

@jit
def compute_distances(x, y):
    n = x.shape[0]
    d = np.empty((n, n))
    for i in range(n):
        for j in range(n):
            d[i, j] = (x[i] - x[j]) ** 2 + (y[i] - y[j]) ** 2
    return d

%timeit d = compute_distances(x, y)
```

```
1.46 ms ± 399 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Bibliografie calcul vectorizat

1. Losing Your Loops: Fast Numerical Computing with NumPy
2. Wes McKinney, *Python for Data Analysis*, ed. a 3-a, cap. 4: NumPy Basics: Arrays and Vectorized Computation
3. Brad Solomon, Look Ma, No for Loops: Array Programming With NumPy, Real Python